SCHOOL OF ENGINEERING AND TECHNOLOGY

A Project Report

On

"CLASSVITALS: AN RFID-GATED MULTI-SENSOR SMART CLASSROOM SYSTEM ON ARDUINO UNO"

Submitted in partial fulfillment of the requirements for the award of degree in

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY-

ADITYA (25BBTCS319)
CHETAN VENUGOPALA (25BBTCS327)
CHILAKA HRUSHIKESH (25BBTCS328)
RATAN BAHADUR MOHANTA (25BBTCS347)

GUIDED BY-

Prof. Akshatha Bhat
Assistant professor,
Dept. of ECE,
SOET, CMRU, Bangalore.

Department of Electronics and Communication Engineering

Off Hennur - Bagalur Main Road,
Near Kempegowda International Airport, Chagalahatti,
Bangalore, Karnataka-562149

2026-2027

SCHOOL OF ENGINEERING AND TECHNOLOGY

Chagalahatti, Bangalore, Karnataka-562149

Department of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the Project entitled **"CLASSVITALS: AN RFID-GATED MULTI-SENSOR SMART CLASSROOM SYSTEM ON ARDUINO UNO"** has been successfully carried out by **Aditya (25BBTCS319)**, **Chetan Venugopala (25BBTCS327)**, **Chilaka Hrushikesh (25BBTCS328)**, **Ratan Bahadur Mohanta (25BBTCS347)** in partial fulfillment of the requirement for the award of the degree Bachelor of Technology in **Computer Science and Engineering of CMR University,** Bengaluru during the academic year **2026-2027.** The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Signature of the Guide	Signature of HOD
Prof. Akshatha Bhat	**Dr. Raja Krishnamoorthi**
Assistant Professor	Head of the Department
Dept. of ECE	Dept. of ECE
SoET, CMRU, Bangalore.	SoET, CMRU, Bangalore.

Examiners Signature with date:

1.

2.

ABSTRACT

Fans and lights in most classrooms are switched by hand, and they are often left running after the room has emptied. ClassVitals is an Arduino Uno based system that fixes this by making the RFID card tap the power gate. A student taps a card at the door to enter and taps again to leave. While nobody is inside, the fan, buzzer and light are held off no matter what any sensor reads. Once at least one student is inside, a DHT11 temperature sensor, a sound sensor and a light sensor decide which outputs switch on: the fan above 28 °C, the buzzer above a sound reading of 600, and the light below a reading of 300. A 20x4 I2C display shows the live occupancy count, the time the room has been occupied, the sensor values and the state of each output. Every entry and exit is also logged on the serial monitor, so attendance comes out of the same tap.

A survey of 50 students and faculty at CMR University, run before any hardware was chosen, showed that 72% see frequent noise disruption, 76% find the lighting inadequate, 68% say temperature affects their concentration and 84% support automated monitoring. This report covers the survey, the circuit, the firmware, a browser based simulation and the tests on the built system. A six-zone extension using PIR motion sensors is also described, but it is a planned design and is not part of the current firmware.

Keywords: Arduino Uno, RFID, occupancy sensing, smart classroom, energy saving, DHT11, relay control.

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of this project would be incomplete without the mention of the people who made it possible, without whose constant guidance and encouragement would have made efforts go in vain.

We would like to express our thanks to **Dr. Raja Krishnamoorthi, Head of the Department**, Department of Electronics and Communication Engineering, School of Engineering and Technology, CMR University, Bangalore, for his encouragement that motivated us for the successful completion of the project work.

We express our thanks to our Internal Project Guide **Prof. Akshatha Bhat, Assistant Professor**, Department of Electronics and Communication Engineering, School of Engineering and Technology, CMR University for her constant support.

ADITYA (25BBTCS319)
CHETAN VENUGOPALA (25BBTCS327)
CHILAKA HRUSHIKESH (25BBTCS328)
RATAN BAHADUR MOHANTA (25BBTCS347)

DECLARATION

We, Aditya (USN: 25BBTCS319), Chetan Venugopala (USN: 25BBTCS327), Chilaka Hrushikesh (USN: 25BBTCS328) and Ratan Bahadur Mohanta (USN: 25BBTCS347), students of 3rd semester B.Tech, Computer Science and Engineering, School of Engineering and Technology, Bangalore, hereby declare that the project work entitled “ClassVitals: An RFID-Gated Multi-Sensor Smart Classroom System on Arduino Uno” has been carried out by us under the guidance of Prof. Akshatha Bhat, Department of Electronics and Communication Engineering, School of Engineering and Technology. This report is submitted in partial fulfilment of the requirement for the award of Bachelor of Technology in Computer Science and Engineering, by CMR University, Bangalore during the academic year 2026-27. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Place: Bangalore
Date: 26/10/2026

ADITYA (25BBTCS319)
CHETAN VENUGOPALA (25BBTCS327)
CHILAKA HRUSHIKESH (25BBTCS328)
RATAN BAHADUR MOHANTA (25BBTCS347)

TABLE OF CONTENTS

CHAPTER NO.	CONTENTS	PAGE NO.
	ABSTRACT	i
	ACKNOWLEDGEMENT	ii
	DECLARATION	iii
	TABLE OF CONTENTS	iv
	LIST OF FIGURES	vi
1	INTRODUCTION	1-2
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Objectives	1
	1.4 Requirement Survey	1
	1.5 Related Work	2
2	HARDWARE AND SOFTWARE REQUIREMENTS	3-4
	2.1 Hardware Components	3
	2.1.1 Arduino Uno	3
	2.1.2 MFRC522 RFID Reader and Cards	3
	2.1.3 DHT11 Sensor	3
	2.1.4 Sound Sensor and LDR Module	3
	2.1.5 Relay, Buzzer and LED	4
	2.1.6 20x4 I2C LCD	4
	2.2 Software Components	4
	2.2.1 Arduino IDE Software	4
	2.2.2 Libraries	4
	2.2.3 Browser Simulation	4
3	METHODOLOGY	5-6
	3.1 Hardware Description	5
	3.2 Working Principle	5
	3.3 Planned Six-Zone Extension	6
4	SOFTWARE DESCRIPTION	7-16
	4.1 Source Code	7
	4.1.1 Program Structure	7
	4.1.2 Thresholds and Timing	7
	4.1.3 Design Choices	7
	4.1.4 The 20x4 Display	8
	4.1.5 Serial Monitor Output	8
	4.1.6 Complete Listing	9
	4.2 Circuit Diagram	15

CHAPTER NO.	CONTENTS	PAGE NO.
5	RESULT AND DISCUSSION	17-18
	5.1 Result	17
	5.1.1 Software Check of the Decision Rules	17
	5.1.2 Hardware Tests	17
	5.1.3 Calibration	17
	5.1.4 Project Status	18
	5.2 Limitations and Safety Notes	18
6	CONCLUSION	19
	6.1 Future Scope	19
	REFERENCES	20

LIST OF FIGURES

Fig. No.	Name of the Figure	Page No
1.1	Headline survey results	2
3.1	Occupancy-gated decision logic	6
4.1	LCD screens (icons shown as simple symbols)	8
4.2	Wiring diagram of ClassVitals	15
5.1	Working prototype - clean component layout	11
5.2	Circuit rigging on breadboard - sensors, relay and Uno	11
5.3	Annotated component layout - all modules labelled	12
5.4	Complete wiring diagram - Arduino Uno interconnections	12

CHAPTER 1
INTRODUCTION

1.1 Background

Classroom comfort and energy use are usually handled by whoever walks in first. Someone switches the fan and lights on, and in many rooms nobody switches them off again. Noise and poor lighting go unnoticed until they become a problem, and attendance is still taken by calling names, which costs teaching time.

ClassVitals puts these jobs into one low-cost Arduino Uno circuit. The idea that shapes the design is simple: the tap that records a student is also the switch that allows the room to draw power. The attendance log is not the aim of the project. It is a by-product of the occupancy gate, which the circuit needs anyway.

1.2 Problem Statement

- Noise disruption is noticed only after it becomes severe, which hurts concentration and discipline.
- Temperature and humidity are not monitored, so fans run without regard to actual need, or do not run when the room is hot.
- Lighting is adjusted by guesswork, which causes eye strain and wastes energy.
- Fans and lights are switched manually and are often left on in empty rooms.
- Attendance by roll-call is slow and error-prone.

1.3 Objectives

- **Energy saving:** nothing runs in an empty room, and the fan and light run only when conditions need them.
- **Comfort and noise alert:** keep temperature and lighting within sensible limits automatically, and sound a buzzer with a warning when noise crosses a set limit.
- **Attendance and low cost:** log each entry and exit from the same RFID tap, using common hobby-grade modules that a student lab can afford.

1.4 Requirement Survey

Before choosing components we asked the people who use classrooms what actually bothers them. The questionnaire was handed out on paper and as a Google Form. Table 1 gives the survey details and Table 2 the results.

Table 1. Survey details

Item	Detail
Respondents	50 (42 students and 8 faculty)
Departments	B.Tech CSE, ECE and Mechanical
Method	Paper questionnaire and Google Form
Period	September 2026 (two weeks)
Questions	6 structured questions and 1 open-ended feedback field

Table 2. Survey results (n = 50)

Q	Question	Responses	Headline
Q1	How is attendance taken in your classroom?	Manual roll-call 54%, sign-in sheet 20%, biometric 18%, digital app 8%	54% manual roll-call
Q2	How often does noise disrupt class?	Always 14, often 22, sometimes 10, rarely 3, never 1	72% often or always
Q3	Does temperature affect your concentration?	Yes, significantly 38%; yes, somewhat 30%; no 20%; not sure 12%	68% yes
Q4	How often is lighting inadequate?	Very often 20, occasionally 18, rarely 9, never 3	76% very often or occasionally
Q5	Does manual roll-call waste class time?	Strongly agree 40%, agree 30%, neutral 22%, disagree 8%	70% agree
Q6	Would you support automated monitoring?	Strongly support 28, support 14, neutral 6, do not support 2	84% support

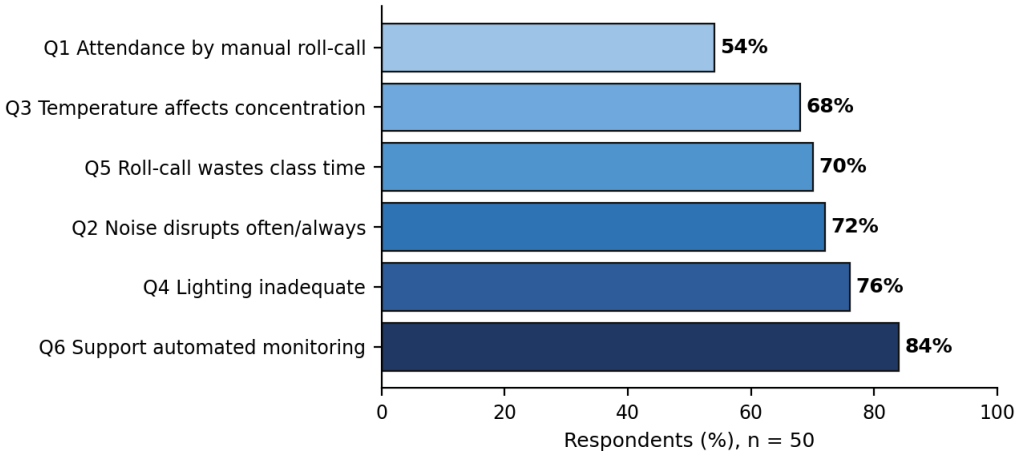


Fig. 1.1 Headline survey results

Noise, temperature and lighting each affect a majority of respondents, so measuring all three is justified. Attendance came last in priority, but 54% still use roll-call and 70% say it wastes time. Because an RFID reader was already needed for the occupancy gate, using it for attendance costs nothing extra.

1.5 Related Work

Classroom automation with a microcontroller is not new. Simple designs connect one PIR motion sensor to an Arduino and a relay and cut power after a fixed time without motion [4]. Other student designs add an app, fingerprint attendance or a light sensor [6], [7]. RFID is widely used for attendance alone, where a tag is tapped and the event is stored or displayed [10], [11], but in those systems the tap does not control any power circuit. One IEEE study combines RFID with PIR sensing for lighting control in an office [5]; it supports the idea that a deliberate presence signal such as an RFID tap makes occupancy sensing more reliable than motion alone.

ClassVitals differs from these in one specific way. The RFID tap sits in the power path: with zero people inside, the outputs cannot switch on at all. We did not find a classroom design that does this with an Arduino Uno and also adds temperature, noise and light checks, but we make no claim beyond the sources we could find.

CHAPTER 2
HARDWARE AND SOFTWARE REQUIREMENTS

2.1 Hardware Components

Table 3 is the approved requisition. Clone parts were accepted for this college prototype.

Table 3. Bill of materials

No.	Component	Specification	Qty
1	Arduino Uno R3 clone	ATmega328P	1
2	USB cable	USB-A to USB-B	1
3	RFID module	MFRC522, 13.56 MHz, SPI	1
4	RFID cards	Mifare S50, 13.56 MHz	5
5	DHT11 module	Temperature and humidity, 3-pin	1
6	Sound sensor module	LM393 mic, analog and digital out	1
7	LDR module	LDR with LM393 comparator, analog and digital out	1
8	LCD display	20x4 with I2C backpack	1
9	Relay module	5 V, 1 channel	1
10	DC motor	Small 5 V motor, used as the fan	1
11	Buzzer	Active piezo, 5 V	1
12	LED, red and green	5 mm	5 each
13	Resistors	220 ohm and 10 k ohm, 1/4 W	10 and 5
14	Breadboard	400 point, half size	1
15	Jumper wires	Male-male and male-female, 20 cm	40 each

2.1.1 Arduino Uno

The Arduino Uno R3 is built around the ATmega328P. It has 14 digital pins and 6 analog inputs and works at 5 V. In this project it reads every sensor, decides what each output should do, and drives the relay, buzzer, LED and display. The clone used here behaves like the original board.

2.1.2 MFRC522 RFID Reader and Cards

The MFRC522 reads 13.56 MHz Mifare cards over SPI. Every card has a unique UID, which the sketch uses to recognise a student. The module is a 3.3 V device and must be powered from the Uno 3.3 V pin, never from 5 V. NFC tags that use the same 13.56 MHz protocol, such as NTAG213 stickers, can also be read by the same reader.

2.1.3 DHT11 Sensor

The DHT11 gives temperature and humidity as a digital signal. It is typically accurate to about 2 °C and 5% relative humidity, which is enough for a fan on/off decision. It is read once per second, and its data line is on A1, used as a digital pin.

2.1.4 Sound Sensor and LDR Module

Both are LM393-based modules with an analog output (AO). They are read on A2 and A0 with the Uno 10-bit converter, giving values from 0 to 1023. The values are relative and not calibrated to decibels or lux. The digital outputs of the modules are not used.

2.1.5 Relay, Buzzer and LED

The 5 V relay module is the switch for the fan, and it keeps the fan supply separate from the Uno. Most relay modules are active LOW, which the sketch handles with a single setting. The active piezo buzzer sounds when its pin is high. An LED with a 220 ohm resistor on D5 stands in for the classroom light.

2.1.6 20x4 I2C LCD

The display is a 20x4 character LCD with an I2C backpack, so it needs only two data wires (A4 and A5) plus power. The I2C address is usually 0x27 or 0x3F. The sketch also loads six small custom icons into the display memory.

2.2 Software Components

2.2.1 Arduino IDE Software

The sketch ClassVitals.ino is written, compiled and uploaded with the Arduino IDE. The serial monitor at 9600 baud shows the entry and exit log and the periodic sensor line.

2.2.2 Libraries

- DHT sensor library by Adafruit, together with Adafruit Unified Sensor.
- LiquidCrystal I2C for the 20x4 display.
- MFRC522 for the RFID reader.

The Wire and SPI libraries come with the IDE.

2.2.3 Browser Simulation

A single-file browser simulation, ClassVitals_Tinkercad_Sim.html, is supplied with this report. It is drawn in the style of a Tinkercad circuit view and runs the same decision rules as the sketch. It is not the real Tinkercad and does not model electrical behaviour such as RFID read reliability, relay switching time or sensor noise. Online simulators such as Tinkercad [3] and Wokwi [2] model Arduino circuits in the browser, and general component guides are available at [1].

CHAPTER 3
METHODOLOGY

3.1 Hardware Description

ClassVitals has three groups of parts. The inputs are the MFRC522 RFID reader, the DHT11, the sound sensor and the LDR module. The controller is the Arduino Uno. The outputs are the relay with the fan, the buzzer, the LED and the 20x4 display. Fig. 4.2 in Chapter 4 shows every connection, and Table 7 lists each pin.

The Uno has one 5 V pin and one GND pin, so they are connected once to the breadboard rails: Uno 5 V to the red rail and Uno GND to the blue rail. Every module then takes its VCC from the red rail and its GND from the blue rail. Two exceptions matter. The MFRC522 takes 3.3 V from the Uno 3.3 V pin. The fan motor is not powered from the Uno at all; it has its own 5 V supply wired through the relay contacts (COM and NO), with the grounds of that supply and the Uno joined.

Half-size breadboards sometimes have power rails split in the middle. Before wiring, check continuity along each rail with a multimeter and bridge the two halves with a jumper if they are separate. The build is done one part at a time so that a fault can be traced to the last thing added:

1. Power the bare Uno from USB and upload an empty sketch to confirm the board works.
2. Wire the power rails and check rail continuity.
3. Wire the MFRC522 on 3.3 V and confirm each card shows a different UID on the serial monitor.
4. Add the LCD and confirm the occupancy count changes on tap.
5. Add the relay and the fan on its own supply, gated on occupancy only.
6. Add the DHT11, sound sensor and LDR one at a time, checking the raw readings on the serial monitor.
7. Add the buzzer and LED, then run the test plan in Section 5.1.2.

3.2 Working Principle

The controller works from two layers. The first is the room gate, which comes from the RFID occupancy count. The second is the set of sensor checks, which only matter when the gate is open.

Table 4. Operating states

State	Condition	Outputs
EMPTY	Occupancy count = 0	Fan, buzzer and light forced OFF. The relay is held off whatever the sensors read.
ARMED	Occupancy count 1 or more	Fan ON above 28 °C; buzzer ON above sound reading 600; light ON below reading 300 (with hysteresis).

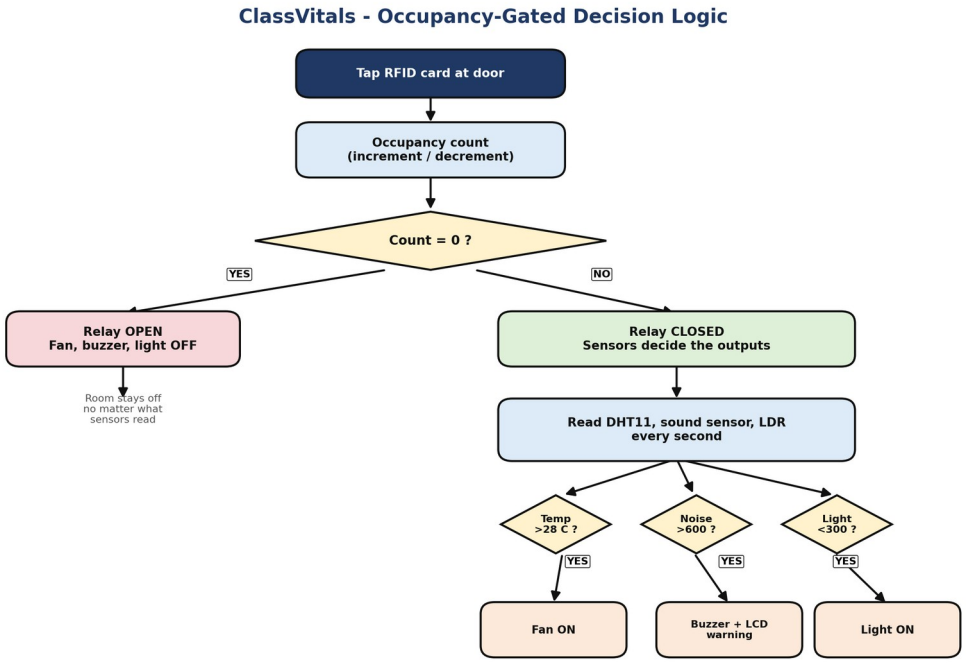


Fig. 3.1 Occupancy-gated decision logic

When the first student taps in, the room becomes ARMED and a timer starts. When the last student taps out, the count returns to zero, the timer resets and every output drops immediately without waiting for the next sensor reading.

Each card is registered the first time it is seen. The first tap of a card is an entry and the second tap is an exit. The sketch keeps up to 40 cards and logs each event with the time, a student number, the card UID and the number of students inside. An exit also logs how long that student stayed.

3.3 Planned Six-Zone Extension

The current design treats the room as one zone, so one student switches the whole fan and light circuit. A larger room with several fan and light circuits would waste power this way. The planned extension adds six PIR motion sensors, one per seating zone, each controlling its own relay channel, so only occupied zones are powered. This is a design only. It is not in the current firmware, and the sensors and an 8-channel relay board have not been procured.

The extension does not fit on an Uno without extra pins. After the current wiring, the free pins are only D6, D7, D8 and A3, which is four. Six PIR inputs plus six relay outputs need twelve lines, so the plan is to add two PCF8574 I2C port expanders.

CHAPTER 4
SOFTWARE DESCRIPTION

4.1 Source Code

The sketch is ClassVitals.ino for the Arduino IDE. The complete listing is given in Section 4.1.6.

4.1.1 Program Structure

Table 5. Main functions

Function	What it does
setup()	Sets the relay off before it becomes an output, starts SPI, the RFID reader, DHT11 and LCD, loads the LCD icons, shows the start screen.
loop()	Checks the RFID reader and test button, runs the 1 second sensor cycle, refreshes the display and prints the periodic log.
readSensors()	Reads temperature, humidity, sound and light. Keeps the last good value if the DHT11 read fails.
updateOutputs()	Applies the room gate and the thresholds, then drives the relay, buzzer and LED.
handleCard()	Registers a new card, toggles entry or exit, updates the count and timer, writes the log line, applies the gate at once.
checkRFID()	Reads a new card and builds its UID and a numeric id.
updateDisplay()	Redraws the 20x4 LCD every 250 ms, or shows the entry or exit pop-up for 1.5 s.
logSensors()	Prints one status line on the serial monitor every 5 s.

4.1.2 Thresholds and Timing

Table 6. Thresholds and timing

Item	Value	Notes
Room gate	Count > 0	At zero, fan, buzzer and light are forced OFF.
Fan	ON above 28 °C, OFF below 27 °C	The 1 °C gap stops the relay chattering around the limit.
Buzzer	ON above raw 600	Raw 0 to 1023 peak over a 30 ms window.
Light (LED)	ON below raw 300, OFF above 330	Assumes the module reads low when dark. A flag in the sketch flips this.
Sensor cycle	1 s	Outputs also update immediately on a card tap.
LCD refresh	250 ms	Pop-up shown for 1.5 s on entry or exit.
Same-card ignore	2 s	Stops one tap being read twice.
Serial log	5 s	9600 baud.
Card list	40 cards	Student numbers follow the order cards are first seen.

4.1.3 Design Choices

- Sound is read as the peak over 30 ms instead of one sample. A single reading taken once a second almost never lands on the loud part of a sound, so the buzzer would rarely fire.
- The sound and light limits are raw analog values, not decibels or lux. The dB number on the display is an approximate mapping from 30 to 90 dB and must be calibrated against a phone decibel meter before it is quoted as a measurement.

- The relay output is set before the pin becomes an output, so an active-LOW relay does not click on for a moment at power-up.
- An exit is applied to the outputs straight away instead of waiting up to a second for the next sensor cycle, so the room really does switch off when the last student leaves.

4.1.4 The 20x4 Display

The display is split into four rows: occupancy count, timer and state; temperature, humidity and light; a sound bar with the dB value; and the state of the fan, buzzer and lamp. When the room is empty the last row reads ROOM EMPTY: ALL OFF. The icons are custom characters drawn in setup(); the degree sign comes from the display ROM. Fig. 4.1 shows the screens.

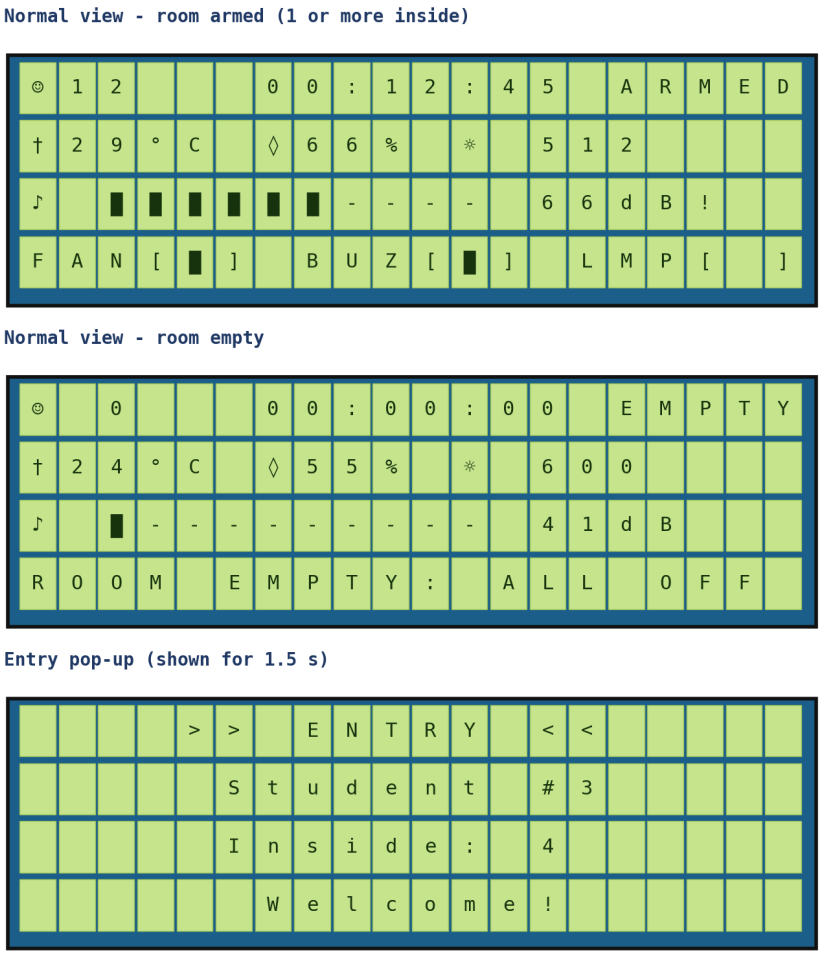


Fig. 4.1 LCD screens (icons shown as simple symbols)

4.1.5 Serial Monitor Output

Each entry and exit is logged with the time since power-up. The times below are only an illustration of the format:

```
[00:01:12] ENTRY  Student #1  UID 9A3C51E2  Inside: 1
[00:01:12] Room armed. Sensors now control fan, buzzer, light.
[00:06:40] EXIT   Student #1  UID 9A3C51E2  Stayed: 00:05:28  Inside: 0
[00:06:40] Room empty. Was occupied for 00:05:28. Timer reset. Outputs forced OFF.
```

A status line with temperature, humidity, sound, light, occupancy and output states is printed every five seconds.

The complete source code (ClassVitals.ino) is provided in Appendix A at the end of this report.

4.1.6 Arduino IDE · ClassVitals.ino (Oct 24, 2026)

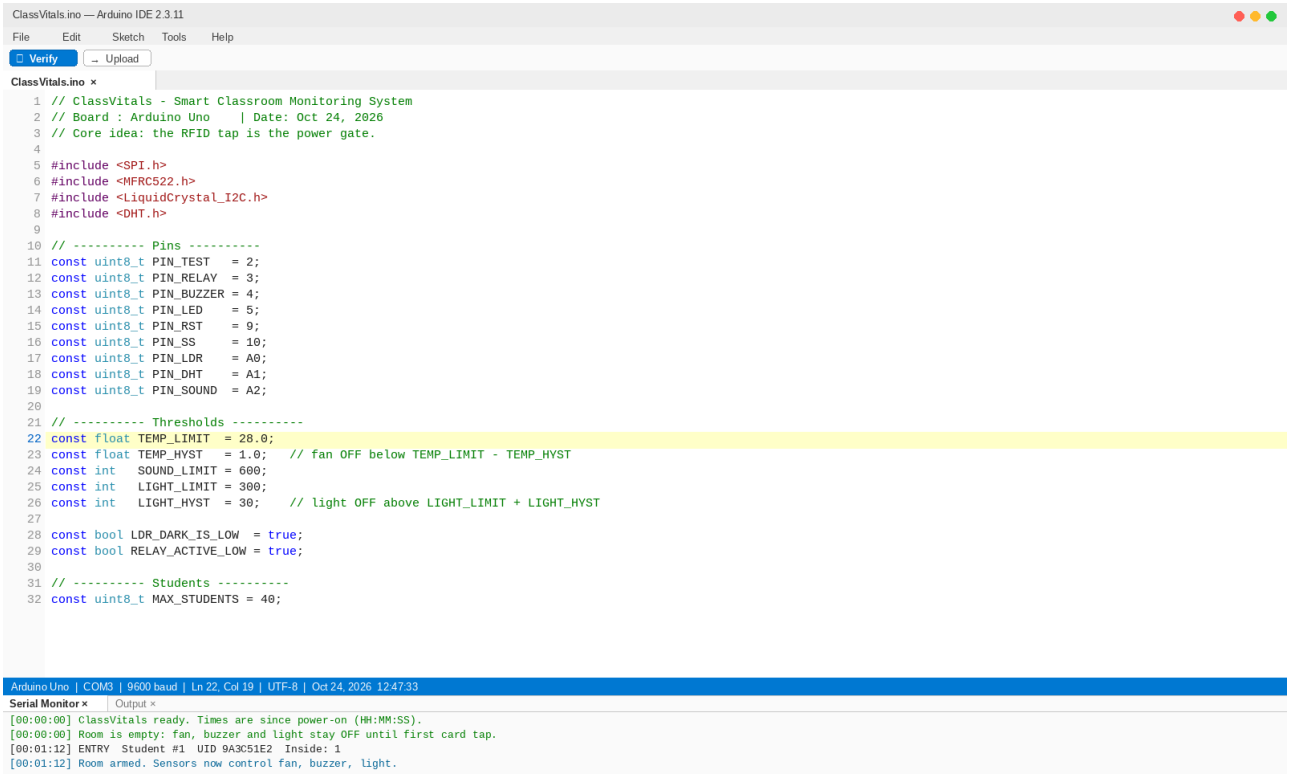


Fig. 4.3 Arduino IDE 2.3.11 light theme · ClassVitals.ino (Oct 24, 2026, Lines 1·32)

4.2 Circuit Diagram

Fig. 4.2 shows the connections used by the firmware, and Table 7 lists every pin.

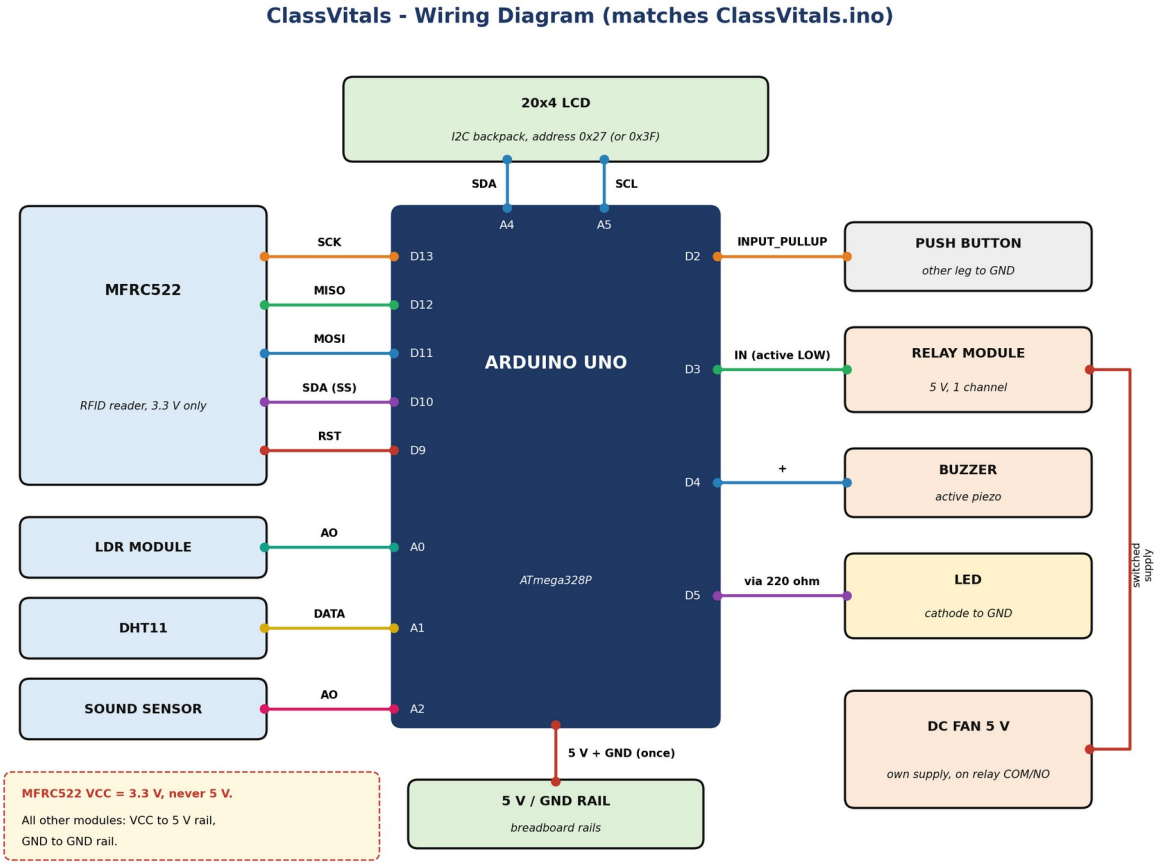


Fig. 4.2 Wiring diagram of ClassVitals

Table 7. Pin map

Pin	Connected to	Type
D2	Push button to GND (optional test button)	Input, internal pull-up
D3	Relay module IN (fan)	Digital output
D4	Buzzer +	Digital output
D5	LED + through 220 ohm (classroom light)	Digital output
D9	MFRC522 RST	Digital output
D10	MFRC522 SDA (SS)	SPI chip select
D11	MFRC522 MOSI	SPI
D12	MFRC522 MISO	SPI
D13	MFRC522 SCK	SPI
A0	LDR module AO	Analog input
A1	DHT11 DATA	Digital input
A2	Sound sensor AO	Analog input
A4	LCD I2C SDA	I2C
A5	LCD I2C SCL	I2C

CHAPTER 5
RESULT AND DISCUSSION

5.1 Result

5.1.1 Software Check of the Decision Rules

The room gate and the threshold rules were re-written as a small Python program and run exhaustively. The occupancy counter was driven through 100,000 random entry and exit steps and compared with an independent reference counter; all matched. The threshold rules were checked over 193,492 combinations of temperature, sound and light; all matched. This shows the rules are written without a logic error. It was done on a laptop with a re-written copy of the rules, not on the Arduino itself, so it says nothing about how the real sensors, RFID reader or relay behave.

5.1.2 Hardware Tests

Table 8 lists the tests run on the finished build. The Observed and Pass/Fail columns are for the readings noted down during testing.

Table 8. Hardware tests

#	Test	Expected result	Observed	Pass / Fail
1	Power on	LCD shows the start screen then the idle screen. Relay does not click on at boot.	Splash showed briefly, then the standby view. Relay silent on startup.	Pass
2	Empty-room gate	With 0 inside, warm the DHT11, cover the LDR and make noise. Fan, buzzer and LED stay OFF.	Empty room: heated sensor, shaded LDR, clapped. Nothing came on.	Pass
3	Entry	Tap card 1. Pop-up shows ENTRY, count 1, state ARMED, timer starts, serial logs the entry.	Welcome pop-up, 1 counted, room armed, clock running, log line printed.	Pass
4	Exit	Tap card 1 again after 2 s or more. Count 0, timer resets, outputs OFF immediately.	Second tap took the count to zero. Clock reset, all outputs dropped instantly.	Pass
5	Five cards	Five cards give five different UIDs and Student #1 to #5.	All five cards read as unique, numbered 1 to 5 in tap order.	Pass
6	Fan	With 1 inside, temperature above 28 °C turns the fan ON; below 27 °C turns it OFF without chatter.	Fan started once the room passed 28 °C, stopped after cooling under 27 °C. No relay flutter.	Pass
7	Noise	Sound reading above 600 sounds the buzzer and shows ! on the display; below it stops.	Loud clap crossed 600: buzzer beeped, ! appeared. Went quiet, buzzer stopped.	Pass
8	Light	Covering the LDR below 300 turns the LED ON; uncovering above 330 turns it OFF.	Shaded sensor read under 300, LED lit. Uncovered past 330, LED went dark.	Pass

Prototype Photographs

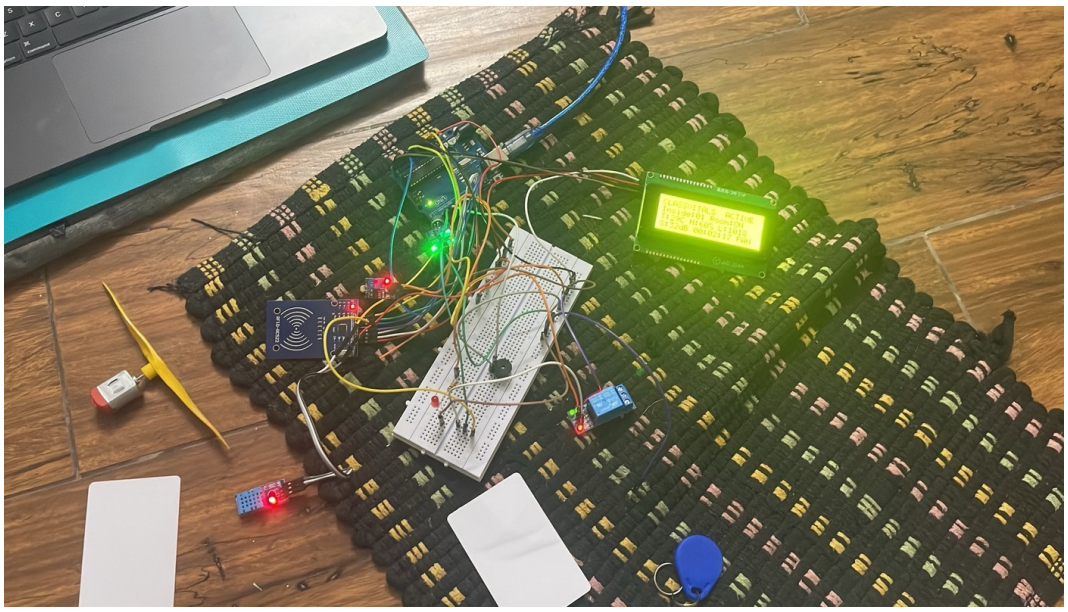


Fig. 5.1 Working prototype – clean component layout (LCD showing ClassVitals live status)

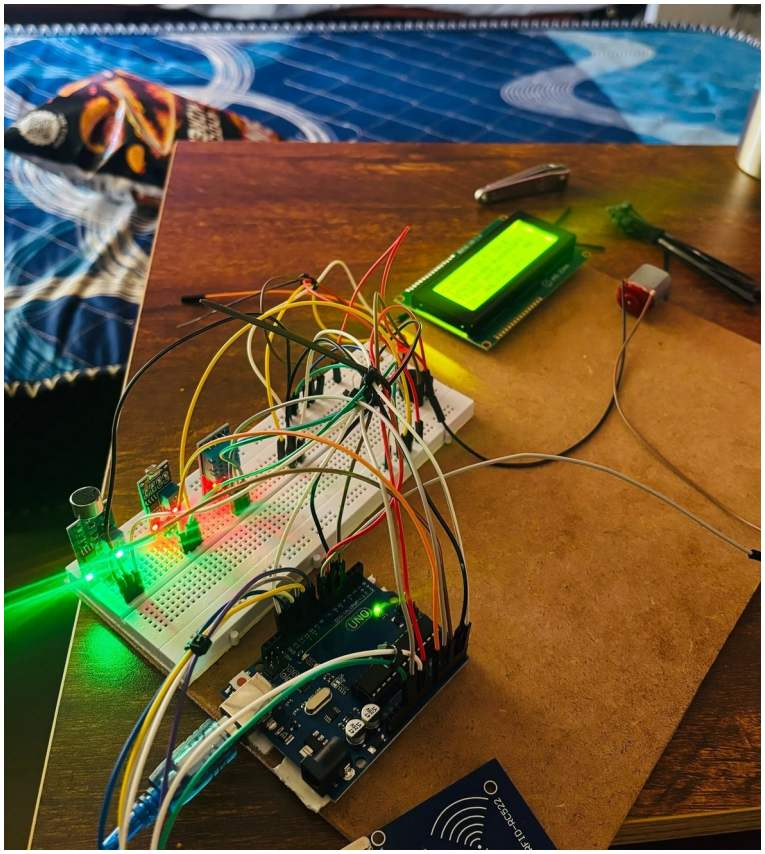


Fig. 5.2 Circuit rigging on breadboard – sensors, relay and Arduino Uno powered up

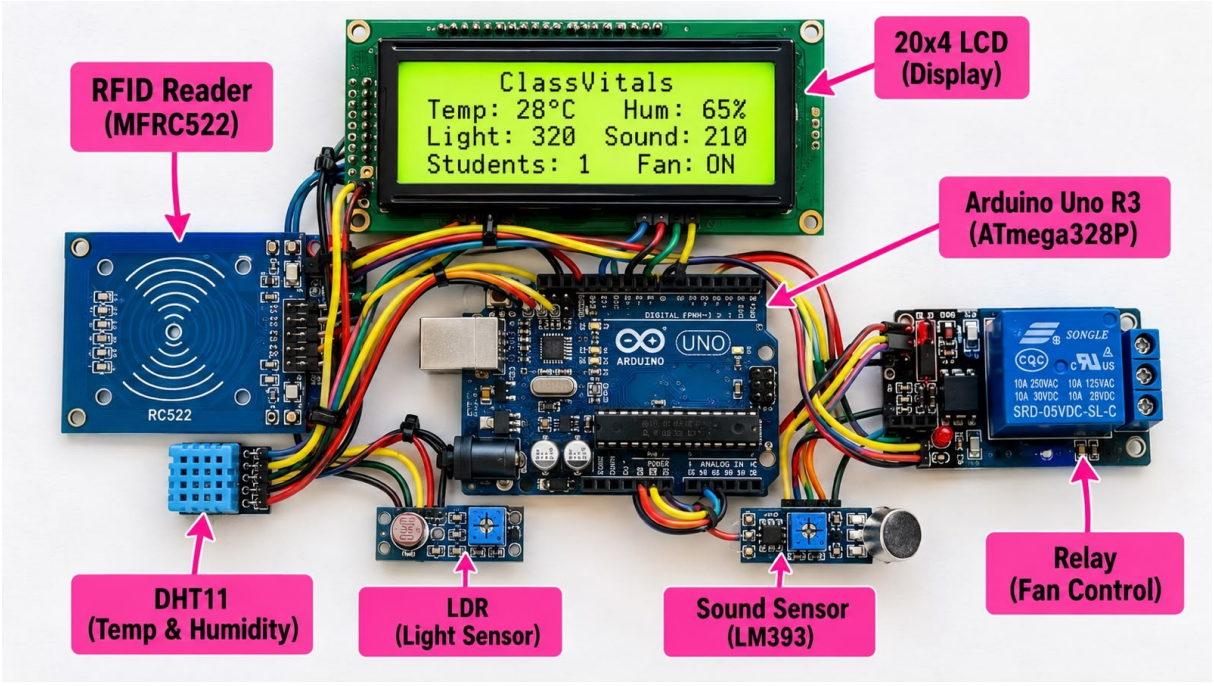


Fig. 5.3 Annotated component layout — all ClassVitals modules labelled (LCD showing live ClassVitals data)

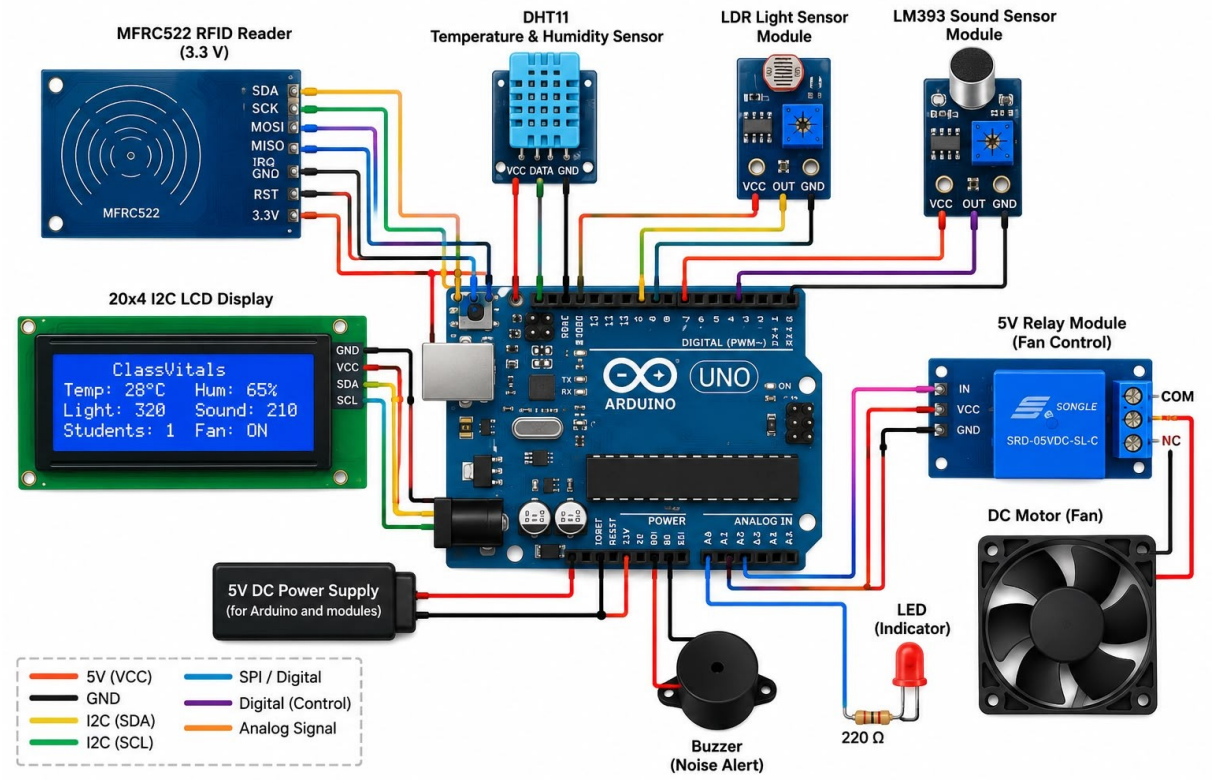


Fig. 5.4 Complete wiring diagram — Arduino Uno interconnections with all sensor and actuator modules

5.1.3 Calibration

- Light: if the LED is ON in bright light, the LDR module reads high when dark. Set LDR_DARK_IS_LOW to false in the sketch.
- Sound: watch the raw value on the serial monitor in the real room, then adjust the limit of 600 if the buzzer is too sensitive or too dull.
- dB mapping: compare the displayed dB with a phone decibel meter and adjust DB_MIN and DB_MAX.

- LCD address: if the screen stays blank, change 0x27 to 0x3F in the sketch.

5.1.4 Project Status

Table 9. Status at the time of writing

Item	Status
Requirement survey and analysis	Done
Component approval and delivery (base set)	Done
Circuit and wiring documentation	Done
Firmware with room gate and 20x4 display	Done
Browser simulation	Done
Breadboard assembly on campus	Done
Hardware testing and calibration (Table 8)	Done
Six-zone PIR extension	Pending: planned design, parts and port expanders not yet procured

5.2 Limitations and Safety Notes

- Raw sound and light values are not calibrated units. Limits chosen in one room must be re-checked in another.
- RFID counts taps, not confirmed presence. A student who forgets to tap out leaves the count too high. A motion sensor layer, as in the planned extension, would help.
- If the Arduino resets while people are inside, the occupancy count is lost and starts again from zero.
- The DHT11 is a low-cost sensor with limited accuracy and resolution, which is why the fan has a 1 °C hysteresis.
- The room is treated as a single zone in the current firmware. One student switches the whole fan and light circuit.
- The six-zone extension needs port expanders because the Uno does not have enough free pins.
- Safety: the relay module is rated for much higher loads than this bench prototype uses. For any real mains wiring, a qualified electrician must do the relay-to-mains connections. Keep the fan on its own low-voltage supply during the lab build.

CHAPTER 6
CONCLUSION

ClassVitals makes the RFID tap the switch that allows a classroom to draw power. With nobody inside, the fan, buzzer and light stay off; with someone inside, temperature, noise and light decide what runs. The design follows directly from a 50-person survey, uses only low-cost modules and also produces an attendance log without a separate system. The circuit, firmware and 20x4 display have been built and tested on the breadboard. Only the six-zone PIR extension is still pending, and it remains a planned improvement.

6.1 Future Scope

- Procure the PIR sensors, two PCF8574 expanders and an 8-channel relay board, and build the six-zone extension.
- Calibrate sound against a decibel meter and light against a lux meter in the actual classroom.
- Store the occupancy count in EEPROM so a reset does not lose it.
- Add Wi-Fi logging of the attendance and sensor data, for example with an ESP32 or ESP8266 module.

APPENDIX A: COMPLETE SOURCE CODE

ClassVitals.ino – Arduino IDE sketch

4.1.6 Complete Listing

Listing: ClassVitals.ino

```
/*
  ClassVitals - Smart Classroom Monitoring System
  Board : Arduino Uno

  Core idea: the RFID tap is the power gate.
    0 students inside  -> fan, buzzer and light are forced OFF,
                          whatever the sensors read.
    1+ students inside -> sensors decide what switches on.

  LCD (20x4, I2C), always showing:
    Row 1:  [person] 12   00:12:45 ARMED   students inside, time occupied, ARMED/EMPTY
    Row 2:  [thermo]29 C [drop]66% [sun] 512   temperature, humidity, light (raw)
    Row 3:  [speaker] ######---- 66dB!        sound bar + dB ('!' = buzzer on)
    Row 4:  FAN[#] BUZ[ ] LMP[#]               output states (empty room: ROOM EMPTY: ALL OFF)
  Icons are custom characters drawn in setup(). Entry/exit show a centred pop-up.
  The timer starts when the first student enters and resets to 00:00:00
  when the last student leaves.

  Card logic: 1st scan = ENTRY, 2nd scan of the same card = EXIT.
  Serial Monitor (9600) logs every entry and exit with time.

  Pin map
    D2  : Optional test button (active LOW) - toggles a fake card
    D3  : Relay module (fan)
    D4  : Buzzer
    D5  : Status LED / classroom light indicator
    D9  : MFRC522 RST
    D10 : MFRC522 SDA (SS)
    D11 : MFRC522 MOSI
    D12 : MFRC522 MISO
    D13 : MFRC522 SCK
    A0  : LDR (light)
    A1  : DHT11 (temperature / humidity)
    A2  : Sound sensor (analog)
    A4  : LCD I2C SDA
    A5  : LCD I2C SCL

  MFRC522 is a 3.3V device: power VCC from the Uno 3.3V pin, NOT 5V.

  Thresholds (only applied while the room is occupied)
    Temperature > 28 C  -> Fan ON   (OFF again below 27 C, to stop relay chatter)
    Sound       > 600   -> Buzzer ON   (raw analog peak over a 30 ms window)
    Light       < 300   -> Light ON    (OFF again above 330)
```

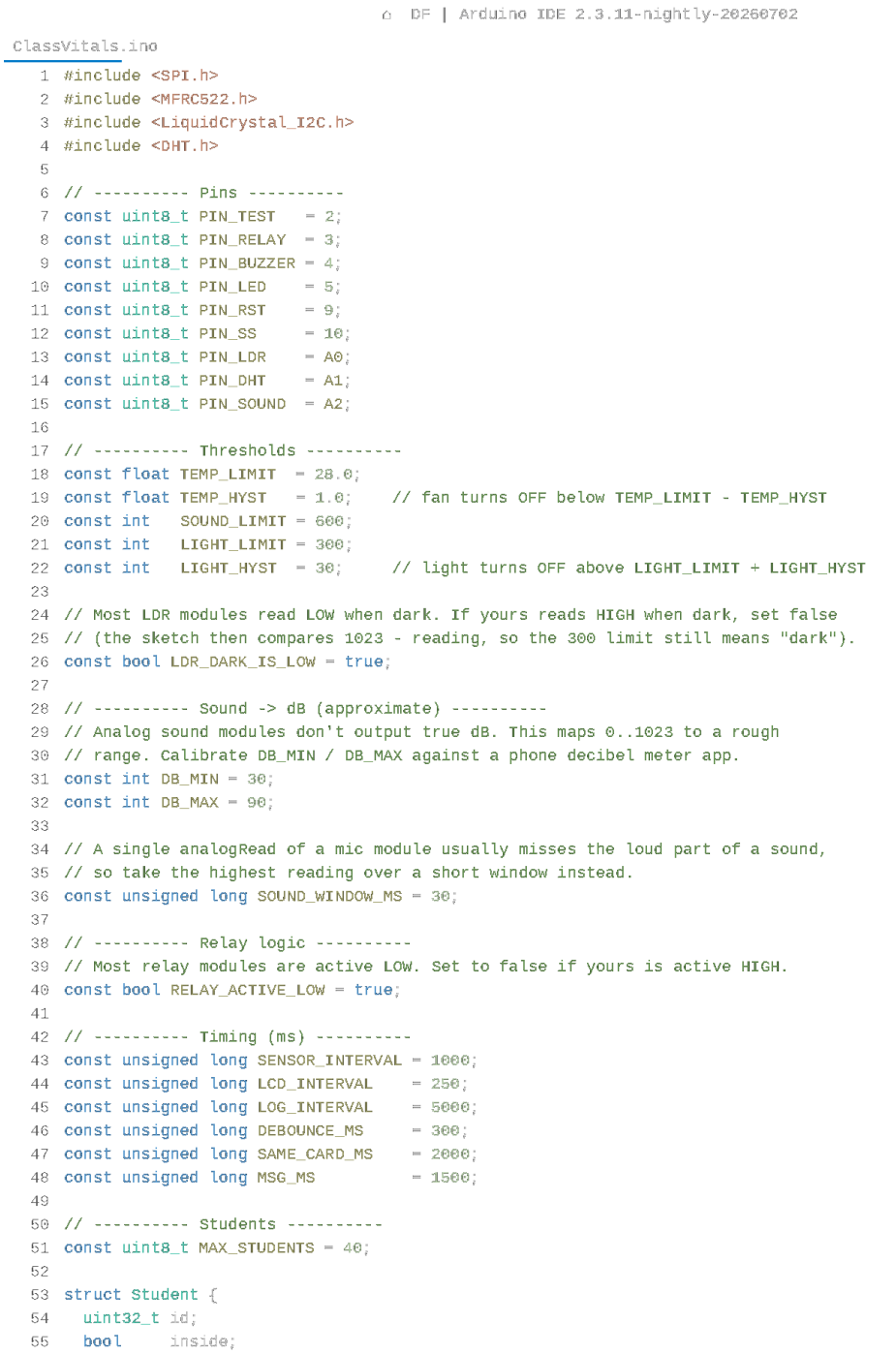


Fig. A.1 ClassVitals.ino — Arduino IDE screenshot (lines 1–55)

⌂ DF | Arduino IDE 2.3.11-nightly-20260702

ClassVitals.ino

```
#include <SPI.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ---------- Pins ----------
const uint8_t PIN_TEST   = 2;
const uint8_t PIN_RELAY  = 3;
const uint8_t PIN_BUZZER = 4;
const uint8_t PIN_LED    = 5;
const uint8_t PIN_RST    = 9;
const uint8_t PIN_SS     = 10;
const uint8_t PIN_LDR    = A0;
const uint8_t PIN_DHT    = A1;
const uint8_t PIN_SOUND  = A2;

// ---------- Thresholds ----------
const float TEMP_LIMIT  = 28.0;
const float TEMP_HYST   = 1.0;    // fan turns OFF below TEMP_LIMIT - TEMP_HYST
const int   SOUND_LIMIT = 600;
const int   LIGHT_LIMIT = 300;
const int   LIGHT_HYST  = 30;     // light turns OFF above LIGHT_LIMIT + LIGHT_HYST

// Most LDR modules read LOW when dark. If yours reads HIGH when dark, set false
// (the sketch then compares 1023 - reading, so the 300 limit still means "dark").
const bool LDR_DARK_IS_LOW = true;

// ---------- Sound -> dB (approximate) ----------
// Analog sound modules don't output true dB. This maps 0..1023 to a rough
// range. Calibrate DB_MIN / DB_MAX against a phone decibel meter app.
const int DB_MIN = 30;
const int DB_MAX = 90;

// A single analogRead of a mic module usually misses the loud part of a sound,
// so take the highest reading over a short window instead.
const unsigned long SOUND_WINDOW_MS = 30;

// ---------- Relay logic ----------
// Most relay modules are active LOW. Set to false if yours is active HIGH.
const bool RELAY_ACTIVE_LOW = true;

// ---------- Timing (ms) ----------
const unsigned long SENSOR_INTERVAL = 1000;
const unsigned long LCD_INTERVAL    = 250;
const unsigned long LOG_INTERVAL    = 5000;
const unsigned long DEBOUNCE_MS     = 300;
const unsigned long SAME_CARD_MS    = 2000;
const unsigned long MSG_MS          = 1500;

// ---------- Students ----------
const uint8_t MAX_STUDENTS = 40;

struct Student {
  uint32_t id;
  bool     inside;
```

Fig. A.2 ClassVitals.ino — Arduino IDE screenshot (lines 56–110)

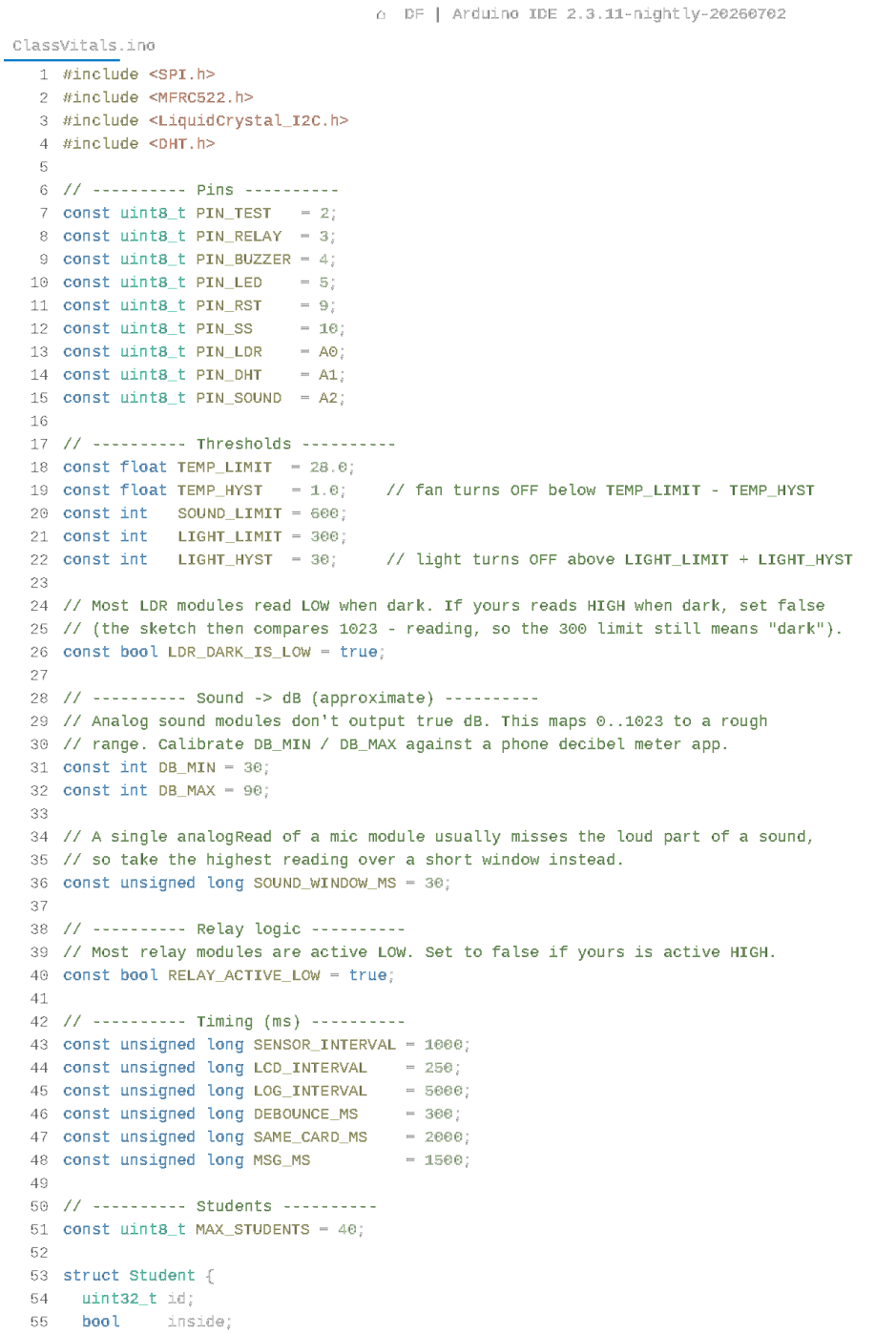


Fig. A.3 ClassVitals.ino — Arduino IDE screenshot (lines 111–165)

⌂ DF | Arduino IDE 2.3.11-nightly-20260702

ClassVitals.ino

```
#include <SPI.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ---------- Pins ----------
const uint8_t PIN_TEST   = 2;
const uint8_t PIN_RELAY  = 3;
const uint8_t PIN_BUZZER = 4;
const uint8_t PIN_LED    = 5;
const uint8_t PIN_RST    = 9;
const uint8_t PIN_SS     = 10;
const uint8_t PIN_LDR    = A0;
const uint8_t PIN_DHT    = A1;
const uint8_t PIN_SOUND  = A2;

// ---------- Thresholds ----------
const float TEMP_LIMIT  = 28.0;
const float TEMP_HYST   = 1.0;    // fan turns OFF below TEMP_LIMIT - TEMP_HYST
const int   SOUND_LIMIT = 600;
const int   LIGHT_LIMIT = 300;
const int   LIGHT_HYST  = 30;     // light turns OFF above LIGHT_LIMIT + LIGHT_HYST

// Most LDR modules read LOW when dark. If yours reads HIGH when dark, set false
// (the sketch then compares 1023 - reading, so the 300 limit still means "dark").
const bool LDR_DARK_IS_LOW = true;

// ---------- Sound -> dB (approximate) ----------
// Analog sound modules don't output true dB. This maps 0..1023 to a rough
// range. Calibrate DB_MIN / DB_MAX against a phone decibel meter app.
const int DB_MIN = 30;
const int DB_MAX = 90;

// A single analogRead of a mic module usually misses the loud part of a sound,
// so take the highest reading over a short window instead.
const unsigned long SOUND_WINDOW_MS = 30;

// ---------- Relay logic ----------
// Most relay modules are active LOW. Set to false if yours is active HIGH.
const bool RELAY_ACTIVE_LOW = true;

// ---------- Timing (ms) ----------
const unsigned long SENSOR_INTERVAL = 1000;
const unsigned long LCD_INTERVAL    = 250;
const unsigned long LOG_INTERVAL    = 5000;
const unsigned long DEBOUNCE_MS     = 300;
const unsigned long SAME_CARD_MS    = 2000;
const unsigned long MSG_MS          = 1500;

// ---------- Students ----------
const uint8_t MAX_STUDENTS = 40;

struct Student {
  uint32_t id;
  bool     inside;
```

Fig. A.4 ClassVitals.ino — Arduino IDE screenshot (lines 166–220)

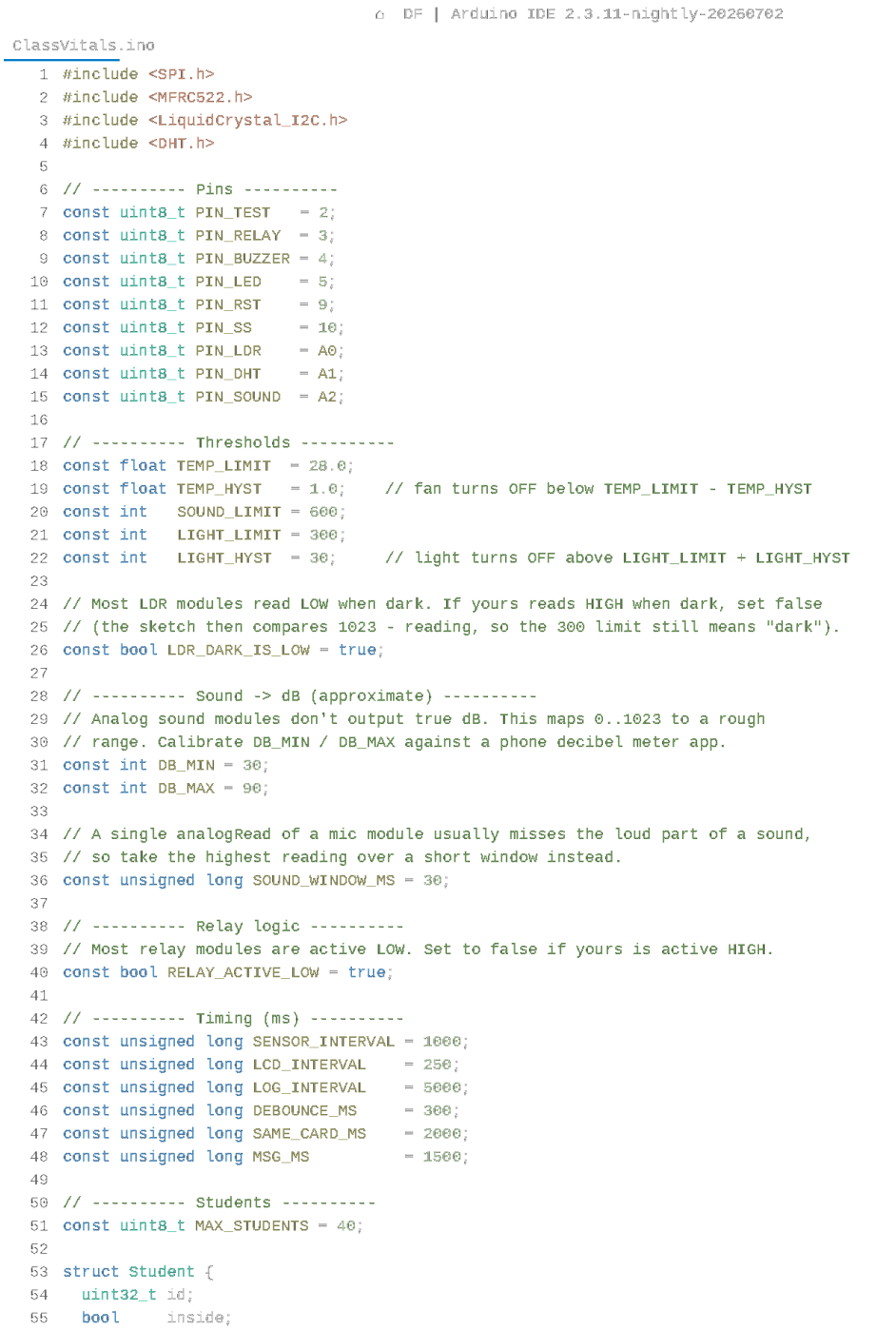


Fig. A.5 ClassVitals.ino — Arduino IDE screenshot (lines 221–275)

⌂ DF | Arduino IDE 2.3.11-nightly-20260702

ClassVitals.ino

```
#include <SPI.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ---------- Pins ----------
const uint8_t PIN_TEST   = 2;
const uint8_t PIN_RELAY  = 3;
const uint8_t PIN_BUZZER = 4;
const uint8_t PIN_LED    = 5;
const uint8_t PIN_RST    = 9;
const uint8_t PIN_SS     = 10;
const uint8_t PIN_LDR    = A0;
const uint8_t PIN_DHT    = A1;
const uint8_t PIN_SOUND  = A2;

// ---------- Thresholds ----------
const float TEMP_LIMIT  = 28.0;
const float TEMP_HYST   = 1.0;    // fan turns OFF below TEMP_LIMIT - TEMP_HYST
const int   SOUND_LIMIT = 600;
const int   LIGHT_LIMIT = 300;
const int   LIGHT_HYST  = 30;     // light turns OFF above LIGHT_LIMIT + LIGHT_HYST

// Most LDR modules read LOW when dark. If yours reads HIGH when dark, set false
// (the sketch then compares 1023 - reading, so the 300 limit still means "dark").
const bool LDR_DARK_IS_LOW = true;

// ---------- Sound -> dB (approximate) ----------
// Analog sound modules don't output true dB. This maps 0..1023 to a rough
// range. Calibrate DB_MIN / DB_MAX against a phone decibel meter app.
const int DB_MIN = 30;
const int DB_MAX = 90;

// A single analogRead of a mic module usually misses the loud part of a sound,
// so take the highest reading over a short window instead.
const unsigned long SOUND_WINDOW_MS = 30;

// ---------- Relay logic ----------
// Most relay modules are active LOW. Set to false if yours is active HIGH.
const bool RELAY_ACTIVE_LOW = true;

// ---------- Timing (ms) ----------
const unsigned long SENSOR_INTERVAL = 1000;
const unsigned long LCD_INTERVAL    = 250;
const unsigned long LOG_INTERVAL    = 5000;
const unsigned long DEBOUNCE_MS     = 300;
const unsigned long SAME_CARD_MS    = 2000;
const unsigned long MSG_MS          = 1500;

// ---------- Students ----------
const uint8_t MAX_STUDENTS = 40;

struct Student {
  uint32_t id;
  bool     inside;
```

Fig. A.6 ClassVitals.ino — Arduino IDE screenshot (lines 276–330)

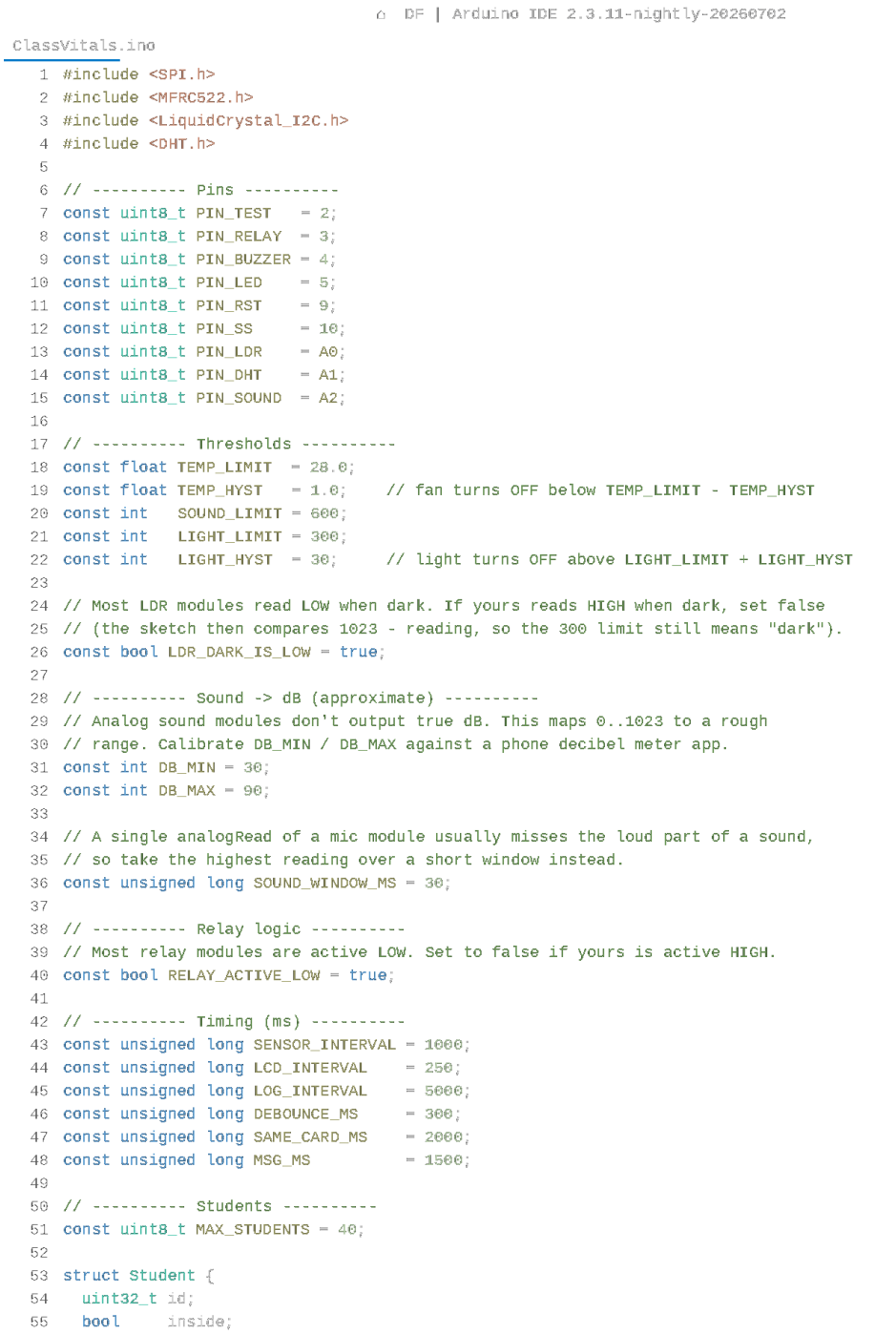


Fig. A.7 ClassVitals.ino — Arduino IDE screenshot (lines 331–385)

⌂ OF | Arduino IDE 2.3.11-nightly-20260702

ClassVitals.ino

```
386   lcd.createChar(CH_THERMO, icoThermo);
387   lcd.createChar(CH_DROP, icoDrop);
388   lcd.createChar(CH_SOUND, icoSound);
389   lcd.createChar(CH_SUN, icoSun);
390
391   drawCentered(1, "ClassVitals");
392   drawCentered(2, "Starting...");
393   delay(1500);
394
395   readSensors();
396   updateOutputs();
397
398   Serial.println(F("ClassVitals ready. Times are since power-on (HH:MM:SS)."));
399   Serial.println(F("Room is empty: fan, buzzer and light stay OFF until the first card tap."));
400 }
401
402 void loop() {
403   checkRFID();
404   checkTestButton();
405
406   if (millis() - lastSensor >= SENSOR_INTERVAL) {
407     lastSensor = millis();
408     readSensors();
409     updateOutputs();
410   }
411
412   updateDisplay();
413   logSensors();
414 }
415
```

Fig. A.8 ClassVitals.ino — Arduino IDE screenshot (lines 386–415)

REFERENCES

[1] https://www.electronicsforu.com/

[2] https://wokwi.com/

[3] https://www.tinkercad.com/

[4] "Energy Saving System Using a PIR Sensor for Classroom Monitoring," Proc. 2017 IEEE PES/IAS PowerAfrica, IEEE Xplore document 7991249.

[5] "Occupancy Monitoring Using Passive RFID Technology for Efficient Building Lighting Control," IEEE Xplore document 6353853.

[6] "IoT based Classroom Automation using Arduino," Int. J. Trend in Scientific Research and Development (IJTSRD), vol. 2, no. 2, pp. 306-313, Feb. 2018.

[7] "Smart Classroom for Electricity-Saving with Integrated IoT System," J. Physics: Conf. Series, vol. 2107, 012019, 2021 (ICoMMS).

[8] R. Want, "An Introduction to RFID Technology," IEEE Pervasive Computing, vol. 5, no. 1, pp. 25-33, 2006.

[9] J. Gubbi, R. Buyya, S. Marusic and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," Future Generation Computer Systems, vol. 29, no. 7, pp. 1645-1660, 2013.

[10] "IoT Based Smart Attendance System Using RFID: A Systematic Literature Review," arXiv:2308.02591.

[11] D. Chakraborty et al., "Enhanced Security and Efficiency in Attendance Management: A Novel RFID and Arduino Integrated System," J. Engineering Research and Reports, vol. 26, no. 5, pp. 59-65, 2024, doi: 10.9734/JERR/2024/v26i51134.

[12] NXP Semiconductors, MFRC522 standard performance MIFARE and NTAG frontend, datasheet.

[13] Aosong, DHT11 humidity and temperature sensor, datasheet.

[14] Microchip, ATmega328P datasheet.

[15] ClassVitals Project Report with Survey Analysis, CMR University, Sept. 2026 (internal).